GEETA UNIVERSITY, PANIPAT

School of Computer Science & Engineering

Session: - 2025-2026

Mobile Aplication Development Lab (CS3741)

Lab Record File

Submitted By: - Rahul (2201301044)

B.Tech CSE 7th Sem (2025-2026)

Submitted To: Mr. Ronak Duggar

	GEETA UNIVERSITY, PANIPAT		
	TOTAL EXPERIMENT: 10	**ISSUE NO:**	**TOTAL PAGES: 25**
SCHOOL OF COMPUTER SCIENCE & ENGINEERING	**LABORATORY: Mobile Aplication Development Lab**	**SEMESTER: 7th**	

S. No	List of Experiment	Page. No	Date	Sign
1.	**Build a Basic React Native Component with State**	1-3		
2.	**Implement a REST API (CRUD) using Node.js, Express, and MongoDB**	4-6		
3.	**Integrate Axios in React Native to Fetch Data from Your Backend**	7-9		
4.	**Create a Notification Demo App (Using Expo Notifications)**	10-11		
5.	**File Storage App using Expo FileSystem**	12-14		
6.	**Build a Location App that Gets and Shares User Location**	15-18		
7.	**Create an Express API with Pagination & Filtering (MongoDB Query Operations)**	19-22		
8.	**Implement a Push Notification System Using Backend Trigger (Expo + Express)**	23-26		
9.	**Develop a Complete MERN Mini App (Frontend + Backend)**	27-28		
10.	**Implement Image Upload from React Native to Express + MongoDB**	29-33		

Experiment - 1

Aim : Create a mobile screen using React Native (Expo) that displays a counter. The counter must increment and decrement using buttons.
Key Focus: Components, useState, styling, layout.

Code :

```
import React, { useState } from 'react';
import { StyleSheet, Text, View, TouchableOpacity, SafeAreaView } from 'react-native';

export default function App() {
  // Initialize state variable 'count' with 0
  const [count, setCount] = useState(0);

  // Function to handle increment
  const handleIncrement = () => {
    setCount(prevCount => prevCount + 1);
  };

  // Function to handle decrement
  const handleDecrement = () => {
    setCount(prevCount => prevCount - 1);
  };

  return (
    <SafeAreaView style={styles.container}>
      <View style={styles.contentContainer}>
        <Text style={styles.title}>Counter App</Text>

        {/* Display the current count */}
        <Text style={styles.counterText}>{count}</Text>

        <View style={styles.buttonContainer}>
          {/* Decrement Button */}
          <TouchableOpacity style={[styles.button, styles.decreaseBtn]} onPress={handleDecrement}>
            <Text style={styles.buttonText}>- Decrease</Text>
          </TouchableOpacity>

          {/* Increment Button */}
          <TouchableOpacity style={[styles.button, styles.increaseBtn]} onPress={handleIncrement}>
            <Text style={styles.buttonText}>+ Increase</Text>
          </TouchableOpacity>
        </View>
      </View>
    </SafeAreaView>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#f5f5f5',
    justifyContent: 'center',
```

```
    alignItems: 'center',
  },
  contentContainer: {
    alignItems: 'center',
    padding: 20,
    backgroundColor: 'white',
    borderRadius: 15,
    shadowColor: '#000',
    shadowOffset: { width: 0, height: 2 },
    shadowOpacity: 0.1,
    shadowRadius: 8,
    elevation: 5,
    width: '80%',
  },
  title: {
    fontSize: 24,
    fontWeight: 'bold',
    marginBottom: 20,
    color: '#333',
  },
  counterText: {
    fontSize: 60,
    fontWeight: 'bold',
    color: '#333',
    marginBottom: 30,
  },
  buttonContainer: {
    flexDirection: 'row',
    justifyContent: 'space-between',
    width: '100%',
  },
  button: {
    paddingVertical: 12,
    paddingHorizontal: 20,
    borderRadius: 8,
    minWidth: 100,
    alignItems: 'center',
  },
  increaseBtn: {
    backgroundColor: '#4CAF50', // Green
  },
  decreaseBtn: {
    backgroundColor: '#f44336', // Red
  },
  buttonText: {
    color: 'white',
    fontSize: 16,
    fontWeight: '600',
  },
});
```

Output :

Experiment - 2

Aim : Create a backend with Express connected to MongoDB that supports CRUD operations for a “Users” collection.
Endpoints: /users (GET, POST), /users/:id (PUT, DELETE)
Submission: Postman collection + source code.

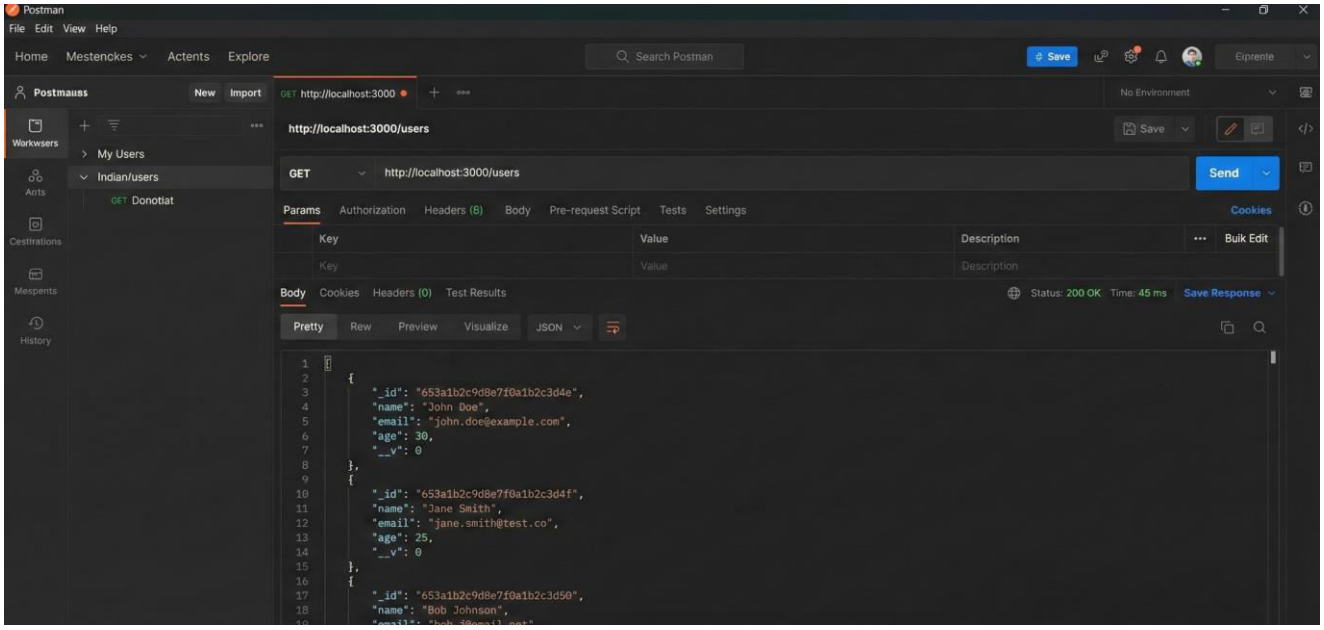
Code :

```
const express = require('express');
const mongoose = require('mongoose');
const bodyParser = require('body-parser');

const app = express();
const PORT = 3000;

// Middleware
app.use(bodyParser.json());

// MongoDB Connection
// Replace with your local URI or Atlas URI
const MONGO_URI = 'mongodb://127.0.0.1:27017/lab_experiment_db';

mongoose.connect(MONGO_URI)
  .then(() => console.log('Connected to MongoDB'))
  .catch(err => console.error('Could not connect to MongoDB', err));

// Mongoose Schema & Model for "Users"
const userSchema = new mongoose.Schema({
  name: String,
  email: String,
  age: Number
});

const User = mongoose.model('User', userSchema);

// --- REST API Routes ---

// 1. CREATE: Add a new User (POST /users)
app.post('/users', async (req, res) => {
  try {
    const newUser = new User(req.body);
    const savedUser = await newUser.save();
    res.status(201).json(savedUser);
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});

// 2. READ: Get all Users (GET /users)
app.get('/users', async (req, res) => {
  try {
    const users = await User.find();
    res.json(users);
```

```
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});

// 3. UPDATE: Update User by ID (PUT /users/:id)
app.put('/users/:id', async (req, res) => {
  try {
    const updatedUser = await User.findByIdAndUpdate(
      req.params.id,
      req.body,
      { new: true } // Returns the updated document
    );
    res.json(updatedUser);
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});

// 4. DELETE: Remove User by ID (DELETE /users/:id)
app.delete('/users/:id', async (req, res) => {
  try {
    await User.findByIdAndDelete(req.params.id);
    res.json({ message: 'User deleted successfully' });
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});

// Start Server
app.listen(PORT, () => {
  console.log(`Server running on http://localhost:${PORT}`);
});
```

- **POST** http://localhost:3000/users: Body {"name": "John", "email": "john@test.com", "age": 25} -> Returns created user object.
- **GET** http://localhost:3000/users: Returns array of user objects.
- **PUT** http://localhost:3000/users/<ID>: Updates specific fields.
- **DELETE** http://localhost:3000/users/<ID>: Removes the user from the database.

Output :


Postman
File Edit View Help
Home Mestenckes Actents Explore
Search Postman
Save
Postmauss
New Import
GET http://localhost:3000
No Environment
Workwsers
Arts
Cestirations
Mespents
History
My Users
Indian/users
GET Donotiat
http://localhost:3000/users
Save
GET
http://localhost:3000/users
Send
Params Authorization Headers (8) Body Pre-request Script Tests Settings
Cookies
Key
Value
Description
Bulk Edit
Body Cookies Headers (0) Test Results
Status: 200 OK Time: 45 ms
Save Response
Pretty Rew Preview Visualize JSON
[
{
"_id": "653a1b2c9d8e7f0a1b2c3d4e",
"name": "John Doe",
"email": "john.doe@example.com",
"age": 30,
"__v": 0
},
{
"_id": "653a1b2c9d8e7f0a1b2c3d4f",
"name": "Jane Smith",
"email": "jane.smith@test.co",
"age": 25,
"__v": 0
},
{
"_id": "653a1b2c9d8e7f0a1b2c3d50",
"name": "Bob Johnson",

Experiment - 3

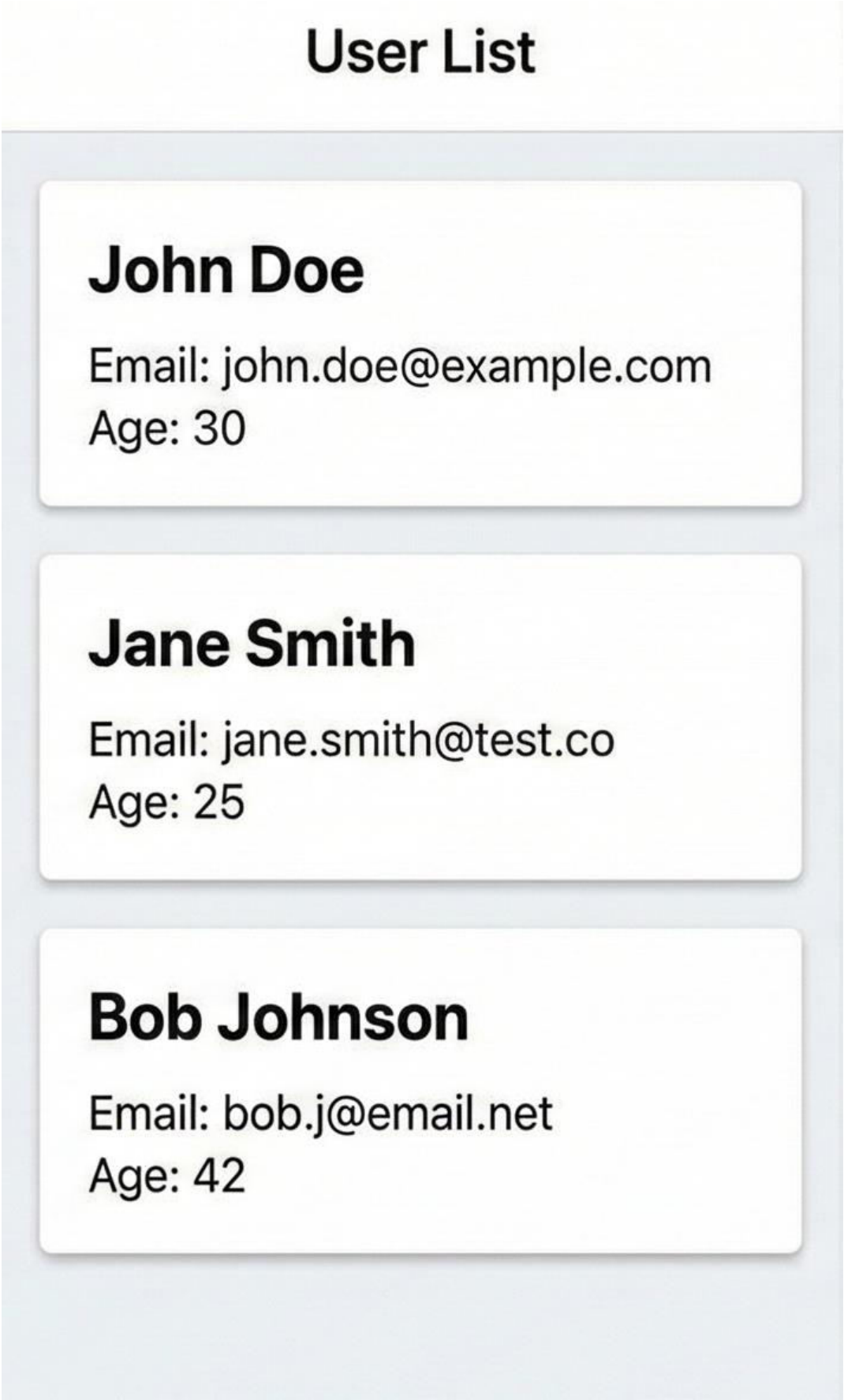
Aim : Build a mobile screen that fetches user data from the API created in Q2 and displays it in a FlatList. Key Focus: Axios integration, useEffect, FlatList rendering.

Code :

```
import React, { useEffect, useState } from 'react';
import { StyleSheet, Text, View, FlatList, ActivityIndicator, SafeAreaView } from 'react-native';
import axios from 'axios';

export default function App() {
  const [users, setUsers] = useState([]);
  const [loading, setLoading] = useState(true);

  // useEffect triggers the fetch function when the component mounts
  useEffect(() => {
    fetchUsers();
  }, []);

  const fetchUsers = async () => {
    try {
      // NOTE: Replace '10.0.2.2' with your computer's local IP address if testing on a physical device.
      // '10.0.2.2' is the special alias for 'localhost' in Android Emulators.
      const response = await axios.get('http://10.0.2.2:3000/users');
      setUsers(response.data);
    } catch (error) {
      console.error("Error fetching data: ", error);
    } finally {
      setLoading(false);
    }
  };

  // The layout for a single row in the list
  const renderItem = ({ item }) => (
    <View style={styles.card}>
      <Text style={styles.name}>{item.name}</Text>
      <Text style={styles.details}>Email: {item.email}</Text>
      <Text style={styles.details}>Age: {item.age}</Text>
    </View>
  );

  return (
    <SafeAreaView style={styles.container}>
      <View style={styles.header}>
        <Text style={styles.headerTitle}>User List</Text>
      </View>

      {loading ? (
        <ActivityIndicator size="large" color="#007bff" style={{ marginTop: 20 }} />
      ) : (
        <FlatList
          data={users}
          keyExtractor={(item) => item._id} // MongoDB uses _id as the unique key
          renderItem={renderItem}
```

```
        contentContainerStyle={styles.listContainer}
      />
    )}
  </SafeAreaView>
);
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#f0f2f5',
  },
  header: {
    padding: 20,
    backgroundColor: '#fff',
    borderBottomWidth: 1,
    borderBottomColor: '#ddd',
    alignItems: 'center',
  },
  headerTitle: {
    fontSize: 22,
    fontWeight: 'bold',
    color: '#333',
  },
  listContainer: {
    padding: 16,
  },
  card: {
    backgroundColor: '#fff',
    padding: 16,
    marginBottom: 12,
    borderRadius: 8,
    // Shadow for iOS
    shadowColor: '#000',
    shadowOffset: { width: 0, height: 2 },
    shadowOpacity: 0.1,
    shadowRadius: 4,
    // Elevation for Android
    elevation: 3,
  },
  name: {
    fontSize: 18,
    fontWeight: 'bold',
    marginBottom: 4,
    color: '#333',
  },
  details: {
    fontSize: 14,
    color: '#666',
    marginBottom: 2,
  },
});
```

Output :


User List
John Doe
Email: john.doe@example.com
Age: 30
Jane Smith
Email: jane.smith@test.co
Age: 25
Bob Johnson
Email: bob.j@email.net
Age: 42

Experiment - 4

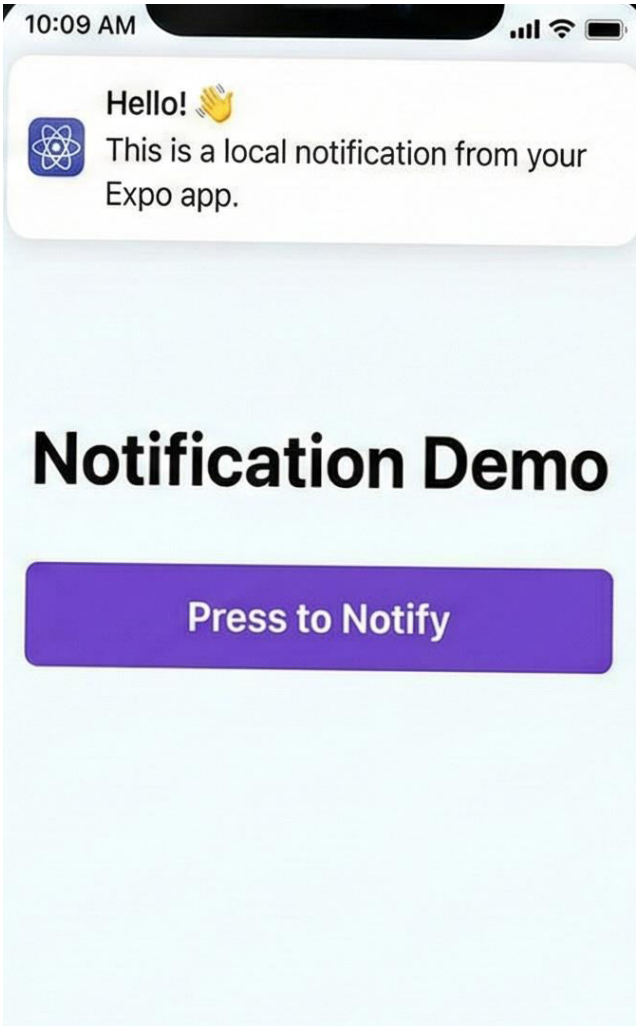
Aim : Implement a React Native app that triggers a local notification when a button is pressed.
Key Focus: Permissions, Notifications API.

Code :

```
import React, { useEffect } from 'react';
import { StyleSheet, Text, View, Button, Alert, Platform } from 'react-native';
import * as Notifications from 'expo-notifications';

// 1. Configure Notification Handler
// This ensures notifications show up even when the app is in the foreground
Notifications.setNotificationHandler({
  handleNotification: async () => ({
    shouldShowAlert: true,
    shouldPlaySound: true,
    shouldSetBadge: false,
  }),
});

export default function App() {

  // 2. Request Permissions on App Start
  useEffect(() => {
    async function requestPermissions() {
      const { status } = await Notifications.getPermissionsAsync();
      if (status !== 'granted') {
        const { status: newStatus } = await Notifications.requestPermissionsAsync();
        if (newStatus !== 'granted') {
          Alert.alert('Permission required', 'Please enable notifications to use this app.');
        }
      }
    }
    requestPermissions();
  }, []);

  // 3. Function to Trigger Notification
  const triggerNotification = async () => {
    await Notifications.scheduleNotificationAsync({
      content: {
        title: "Hello! 👋",
        body: "This is a local notification from your Expo app.",
        data: { data: 'goes here' },
      },
      trigger: { seconds: 1 }, // Trigger after 1 second
    });
  };

  return (
    <View style={styles.container}>
      <Text style={styles.title}>Notification Demo</Text>
      <View style={styles.buttonContainer}>
        <Button
          title="Press to Notify"
```

```
        onPress={triggerNotification}
        color="#6200ee"
      />
    </View>
  </View>
);
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#fff',
    alignItems: 'center',
    justifyContent: 'center',
  },
  title: {
    fontSize: 24,
    fontWeight: 'bold',
    marginBottom: 20,
  },
  buttonContainer: {
    width: '60%',
  }
});
```

Output :


10:09 AM
Hello! 👋
This is a local notification from your Expo app.
Notification Demo
Press to Notify

Experiment - 5

Aim : Design an app that writes a text file to the device's storage and reads it back.
Key Focus: FileSystem API, read/write operations, error handling.

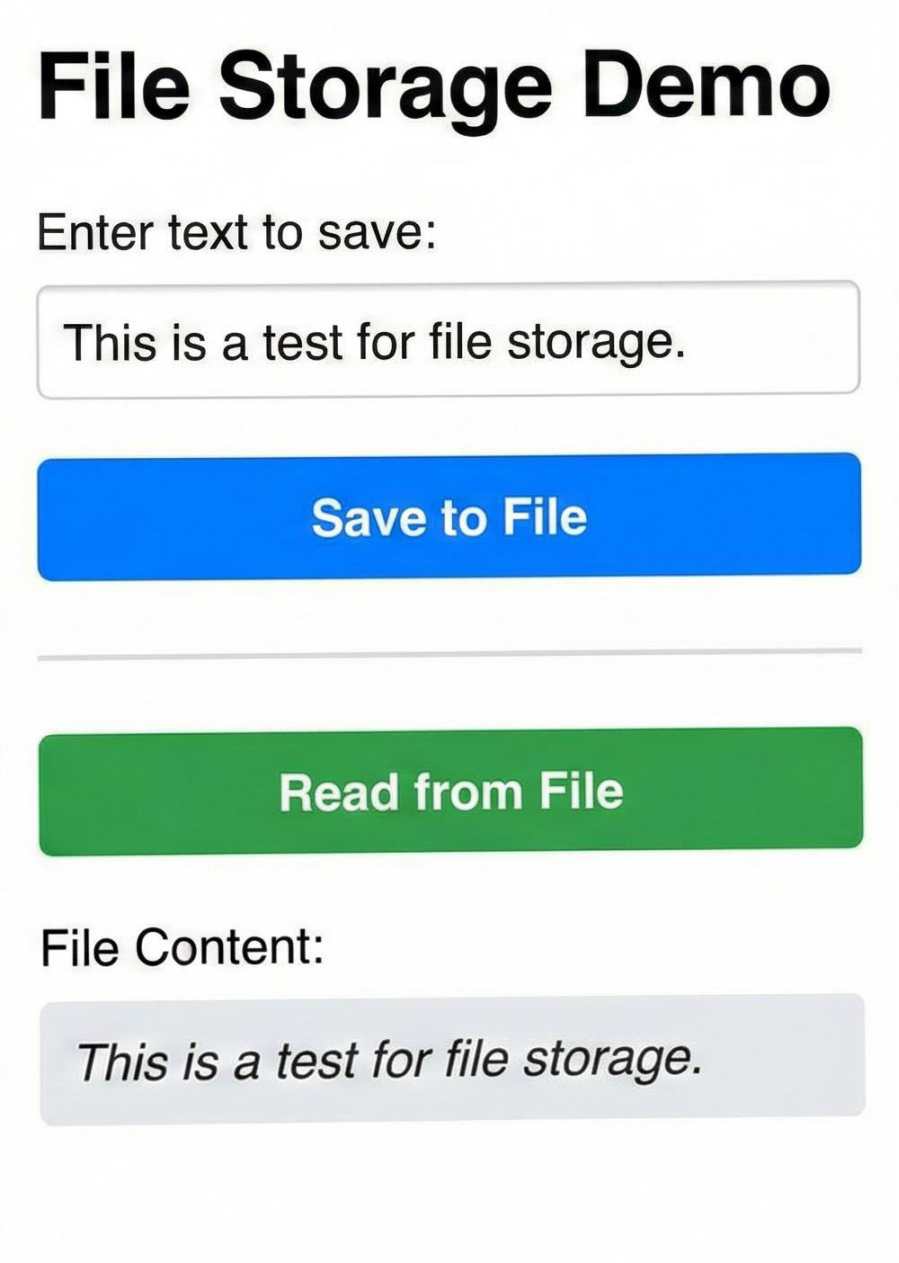
Code :

```
import React, { useState } from 'react';
import { StyleSheet, Text, View, TextInput, Button, Alert, SafeAreaView } from 'react-native';
import * as FileSystem from 'expo-file-system';

export default function App() {
  const [inputText, setInputText] = useState('');
  const [fileContent, setFileContent] = useState('No content read yet.');

  // Define the file URI (Path)
  // We store it in the app's document directory
  const fileUri = FileSystem.documentDirectory + 'my_lab_file.txt';

  // Function to Write to File
  const saveFile = async () => {
    try {
      await FileSystem.writeAsStringAsync(fileUri, inputText);
      Alert.alert('Success', 'File saved successfully!');
    } catch (error) {
      console.error(error);
      Alert.alert('Error', 'Could not save the file.');
    }
  };

  // Function to Read from File
  const readFile = async () => {
    try {
      // Check if file exists before reading
      const info = await FileSystem.getInfoAsync(fileUri);

      if (info.exists) {
        const content = await FileSystem.readAsStringAsync(fileUri);
        setFileContent(content);
      } else {
        Alert.alert('Error', 'File does not exist. Save one first!');
      }
    } catch (error) {
      console.error(error);
      Alert.alert('Error', 'Could not read the file.');
    }
  };

  return (
    <SafeAreaView style={styles.container}>
      <View style={styles.content}>
        <Text style={styles.title}>File Storage Demo</Text>

        {/* Input Area */}
        <Text style={styles.label}>Enter text to save:</Text>
```

```
        <TextInput
          style={styles.input}
          placeholder="Type here..."
          onChangeText={setInputText}
          value={inputText}
        />

        <View style={styles.buttonContainer}>
          <Button title="Save to File" onPress={saveFile} />
        </View>

        <View style={styles.divider} />

        {/* Output Area */}
        <View style={styles.buttonContainer}>
          <Button title="Read from File" color="green" onPress={readFile} />
        </View>

        <Text style={styles.label}>File Content:</Text>
        <Text style={styles.resultBox}>{fileContent}</Text>
      </View>
    </SafeAreaView>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#fff',
    paddingTop: 50,
  },
  content: {
    padding: 20,
  },
  title: {
    fontSize: 24,
    fontWeight: 'bold',
    marginBottom: 20,
    textAlign: 'center',
  },
  label: {
    fontSize: 16,
    fontWeight: '600',
    marginTop: 15,
    marginBottom: 5,
  },
  input: {
    borderWidth: 1,
    borderColor: '#ccc',
    padding: 10,
    borderRadius: 5,
    marginBottom: 15,
  },
  buttonContainer: {
```

```
    marginVertical: 5,
  },
  divider: {
    height: 1,
    backgroundColor: '#ddd',
    marginVertical: 20,
  },
  resultBox: {
    marginTop: 10,
    padding: 15,
    backgroundColor: '#f9f9f9',
    borderRadius: 5,
    borderWidth: 1,
    borderColor: '#eee',
    fontStyle: 'italic',
  },
});
```

Output :


File Storage Demo
Enter text to save:
This is a test for file storage.
Save to File
Read from File
File Content:
This is a test for file storage.

Experiment - 6

Aim : Create a mobile application that retrieves the device's current latitude and longitude and shares the location using the Share API.
Key Focus: Permissions, Location API, Share API.
Submission: Screen output + share functionality.

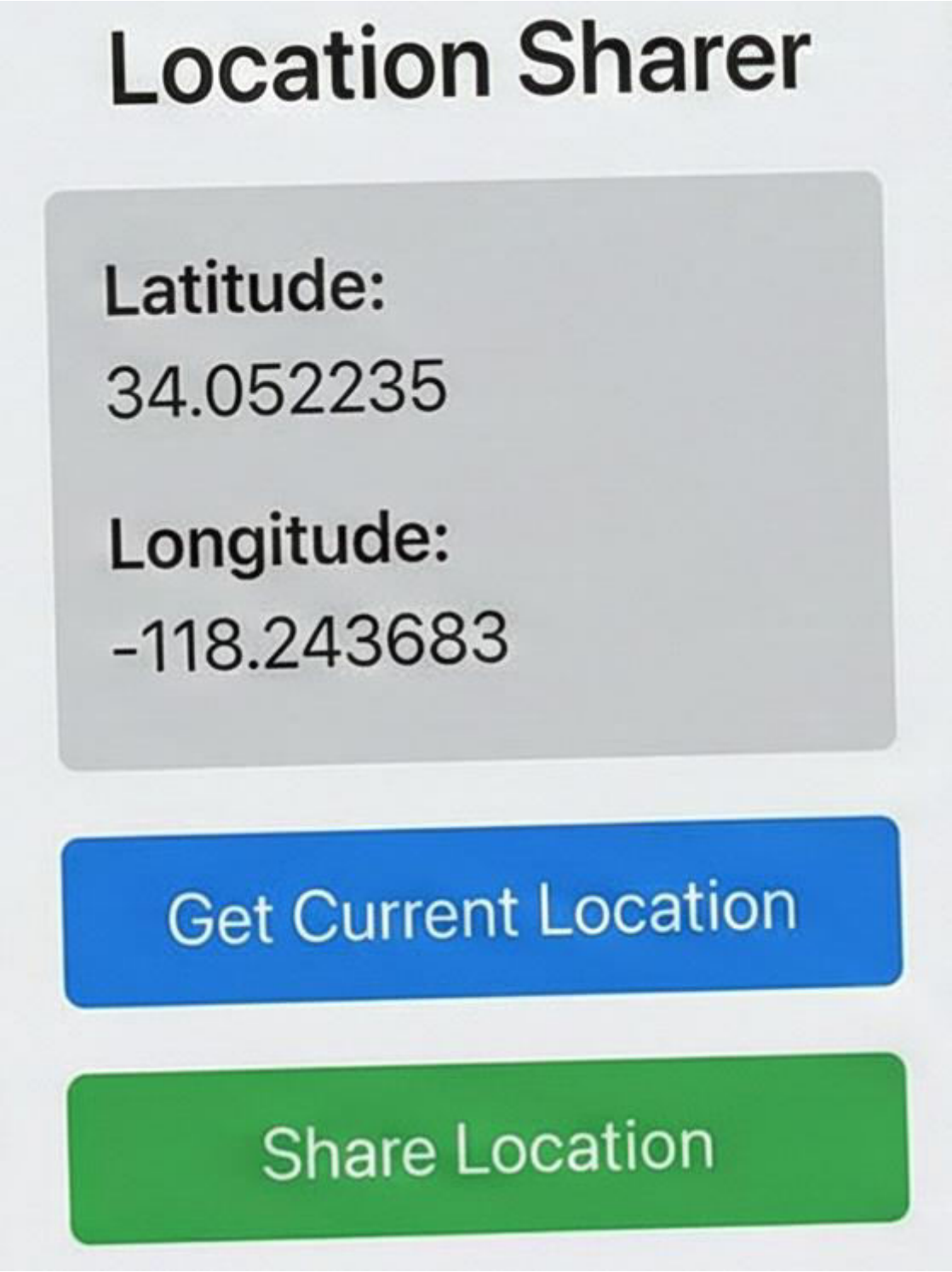
Code :

```
import React, { useState, useEffect } from 'react';
import { StyleSheet, Text, View, Button, Share, ActivityIndicator, Alert, SafeAreaView } from 'react-native';
import * as Location from 'expo-location';

export default function App() {
  const [location, setLocation] = useState(null);
  const [errorMsg, setErrorMsg] = useState(null);
  const [loading, setLoading] = useState(false);

  // 1. Request Permission on Mount
  useEffect(() => {
    (async () => {
      let { status } = await Location.requestForegroundPermissionsAsync();
      if (status !== 'granted') {
        setErrorMsg('Permission to access location was denied');
        return;
      }
    })();
  }, []);

  // 2. Function to Get Location
  const getLocation = async () => {
    setLoading(true);
    try {
      let currentLocation = await Location.getCurrentPositionAsync({});
      setLocation(currentLocation.coords);
      setErrorMsg(null);
    } catch (error) {
      setErrorMsg('Could not fetch location');
    } finally {
      setLoading(false);
    }
  };

  // 3. Function to Share Location
  const shareLocation = async () => {
    if (!location) {
      Alert.alert('No Location', 'Please fetch your location first!');
      return;
    }

    try {
      const mapUrl =
`https://www.google.com/maps/search/?api=1&query=${location.latitude},${location.longitude}`;
      const message = `Here is my current location:\nLatitude: ${location.latitude}\nLongitude:
${location.longitude}\n\nOpen in Maps: ${mapUrl}`;
```

```
      await Share.share({
        message: message,
        title: 'My Location',
      });
    } catch (error) {
      Alert.alert('Error', error.message);
    }
  };

  return (
    <SafeAreaView style={styles.container}>
      <View style={styles.content}>
        <Text style={styles.title}>Location Sharer</Text>

        <View style={styles.infoBox}>
          {loading ? (
            <ActivityIndicator size="large" color="#0000ff" />
          ) : errorMsg ? (
            <Text style={styles.errorText}>{errorMsg}</Text>
          ) : location ? (
            <>
              <Text style={styles.coordLabel}>Latitude:</Text>
              <Text style={styles.coordValue}>{location.latitude}</Text>

              <Text style={styles.coordLabel}>Longitude:</Text>
              <Text style={styles.coordValue}>{location.longitude}</Text>
            </>
          ) : (
            <Text style={styles.placeholder}>Press "Get Location" to start</Text>
          )}
        </View>

        <View style={styles.buttonContainer}>
          <Button title="Get Current Location" onPress={getLocation} />
        </View>

        <View style={styles.buttonContainer}>
          <Button
            title="Share Location"
            color="#4CAF50"
            onPress={shareLocation}
            disabled={!location}
          />
        </View>
      </View>
    </SafeAreaView>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#fff',
```

```
    justifyContent: 'center',
  },
  content: {
    padding: 20,
    alignItems: 'center',
  },
  title: {
    fontSize: 24,
    fontWeight: 'bold',
    marginBottom: 30,
  },
  infoBox: {
    width: '100%',
    padding: 20,
    backgroundColor: '#f0f0f0',
    borderRadius: 10,
    marginBottom: 20,
    alignItems: 'center',
    minHeight: 150,
    justifyContent: 'center',
  },
  coordLabel: {
    fontSize: 16,
    color: '#555',
    marginTop: 5,
  },
  coordValue: {
    fontSize: 20,
    fontWeight: 'bold',
    marginBottom: 10,
    color: '#333',
  },
  errorText: {
    color: 'red',
    textAlign: 'center',
  },
  placeholder: {
    color: '#888',
    fontStyle: 'italic',
  },
  buttonContainer: {
    width: '100%',
    marginVertical: 10,
  },
});
```

Output :


Location Sharer
Latitude:
34.052235
Longitude:
-118.243683
Get Current Location
Share Location

Experiment - 7

Aim : Build a backend API endpoint /products that returns products from MongoDB with:
Pagination (page, limit)
Filtering (category, priceRange, etc.)
Frontend (React Native) must call this API and display results in a paginated list.
Key Focus: MongoDB queries, pagination logic, API integration.

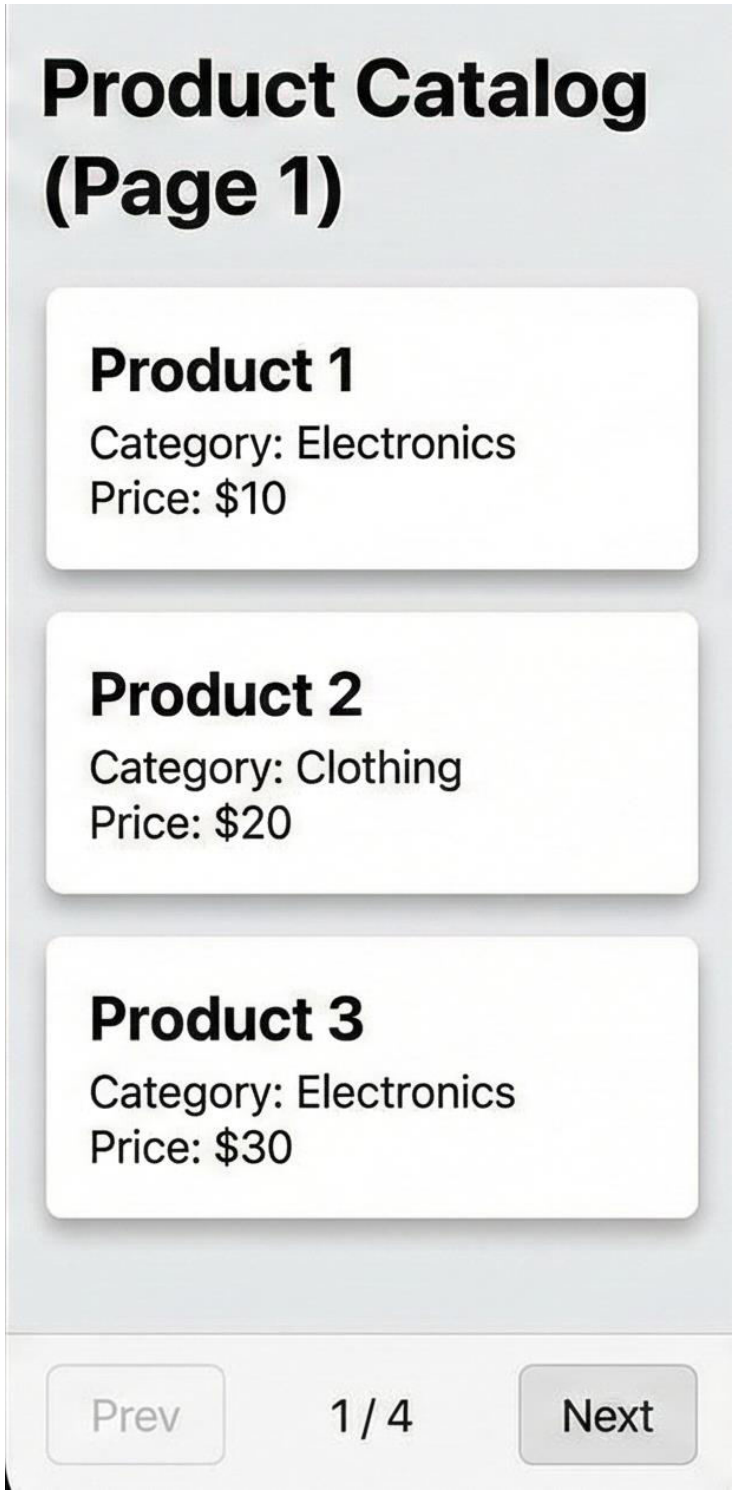
Code :

```
//Server.js

const express = require('express');
const mongoose = require('mongoose');
const cors = require('cors');
const app = express();

app.use(express.json());
app.use(cors());

mongoose.connect('mongodb://127.0.0.1:27017/shop_db')
  .then(() => console.log('Connected to MongoDB'));

// Product Schema
const productSchema = new mongoose.Schema({
  name: String,
  category: String,
  price: Number
});

const Product = mongoose.model('Product', productSchema);

// SEED ROUTE (Run this once to populate DB)
app.post('/seed', async (req, res) => {
  const products = [];
  for(let i=1; i<=20; i++) {
    products.push({
      name: `Product ${i}`,
      category: i % 2 === 0 ? 'Electronics' : 'Clothing',
      price: i * 10
    });
  }
  await Product.insertMany(products);
  res.send('Database Seeded!');
});

// GET /products with Pagination & Filtering
app.get('/products', async (req, res) => {
  try {
    // 1. Destructure query params with defaults
    const { page = 1, limit = 5, category } = req.query;

    // 2. Build Filter Object
    let filter = {};
```

```
    if (category) {
      filter.category = category;
    }

    // 3. Execute Query with Pagination
    const products = await Product.find(filter)
      .limit(limit * 1)          // Convert to number and limit
      .skip((page - 1) * limit)  // Skip previous pages
      .exec();

    // 4. Get total count for UI reference
    const count = await Product.countDocuments(filter);

    res.json({
      products,
      totalPages: Math.ceil(count / limit),
      currentPage: page
    });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

app.listen(3000, () => console.log('Server running on port 3000'));

//App.js

import React, { useState, useEffect } from 'react';
import { StyleSheet, Text, View, FlatList, Button, SafeAreaView } from 'react-native';
import axios from 'axios';

export default function App() {
  const [products, setProducts] = useState([]);
  const [page, setPage] = useState(1);
  const [totalPages, setTotalPages] = useState(1);

  const fetchProducts = async (pageNumber) => {
    try {
      // Replace with your IP address
      const response = await axios.get(`http://10.0.2.2:3000/products?page=${pageNumber}&limit=5&category=Electronics`);
      setProducts(response.data.products);
      setTotalPages(response.data.totalPages);
    } catch (error) {
      console.error(error);
    }
  };

  useEffect(() => {
    fetchProducts(page);
  }, [page]);

  return (
    <SafeAreaView style={styles.container}>
```

```
      <Text style={styles.header}>Product Catalog (Page {page})</Text>

      <FlatList
        data={products}
        keyExtractor={(item) => item._id}
        renderItem={({ item }) => (
          <View style={styles.card}>
            <Text style={styles.name}>{item.name}</Text>
            <Text>Category: {item.category}</Text>
            <Text>Price: ${item.price}</Text>
          </View>
        )}
      />

      <View style={styles.pagination}>
        <Button
          title="Prev"
          disabled={page <= 1}
          onPress={() => setPage(page - 1)}
        />
        <Text style={styles.pageInfo}>{page} / {totalPages}</Text>
        <Button
          title="Next"
          disabled={page >= totalPages}
          onPress={() => setPage(page + 1)}
        />
      </View>
    </SafeAreaView>
  );
}

const styles = StyleSheet.create({
  container: { flex: 1, padding: 20, backgroundColor: '#f5f5f5' },
  header: { fontSize: 22, fontWeight: 'bold', marginBottom: 20, textAlign: 'center' },
  card: { backgroundColor: 'white', padding: 15, marginBottom: 10, borderRadius: 8 },
  name: { fontSize: 18, fontWeight: 'bold' },
  pagination: { flexDirection: 'row', justifyContent: 'space-between', alignItems: 'center', marginTop: 10 },
  pageInfo: { fontSize: 16, fontWeight: 'bold' }
});
```

Output :


Product Catalog (Page 1)
Product 1
Category: Electronics
Price: $10
Product 2
Category: Clothing
Price: $20
Product 3
Category: Electronics
Price: $30
Prev
1 / 4
Next

Experiment - 8

Aim : Create an Express backend API that sends push notifications to a registered Expo push token.
React Native app must:
Register for push notifications
Capture the device's Expo push token
Send it to your backend
Backend must:
Use Expo Push API to send a notification to that device
Key Focus: Notifications, permissions, REST API integration.

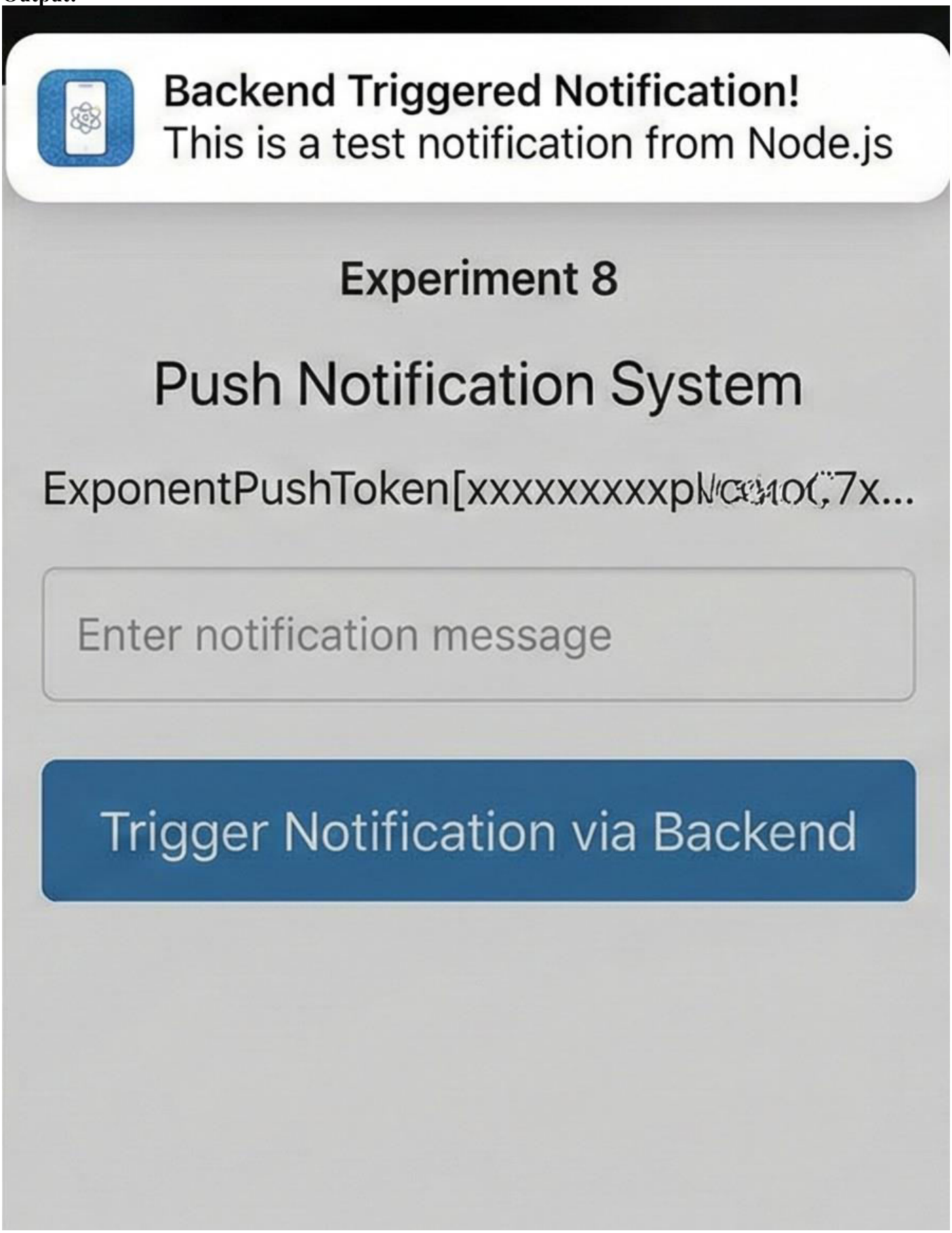
Code :

```
//Server.js

const express = require('express');
const { Expo } = require('expo-server-sdk');
const cors = require('cors');
const bodyParser = require('body-parser');

const app = express();
app.use(cors());
app.use(bodyParser.json());

// Initialize Expo SDK
const expo = new Expo();

// Route to receive notification request
app.post('/send-notification', async (req, res) => {
  const { token, message } = req.body;

  // 1. Validate the token
  if (!Expo.isExpoPushToken(token)) {
    return res.status(400).send({ error: 'Invalid Expo Push Token' });
  }

  // 2. Construct the message
  const messages = [{
    to: token,
    sound: 'default',
    title: 'Backend Triggered Notification!',
    body: message || 'This is a test notification from Node.js',
    data: { someData: 'goes here' },
  }];

  // 3. Send the notification using Expo SDK
  try {
    const chunks = expo.chunkPushNotifications(messages);
    for (const chunk of chunks) {
      await expo.sendPushNotificationsAsync(chunk);
    }
    res.send({ status: 'Notification sent successfully' });
  } catch (error) {
    console.error(error);
```

```
    res.status(500).send({ error: 'Error sending notification' });
  }
});

app.listen(3000, () => console.log('Server running on port 3000'));

//App.js

import React, { useState, useEffect } from 'react';
import { StyleSheet, Text, View, Button, TextInput, Alert, Platform } from 'react-native';
import * as Notifications from 'expo-notifications';
import * as Device from 'expo-device';
import axios from 'axios';

// Configure how notifications appear when app is in foreground
Notifications.setNotificationHandler({
  handleNotification: async () => ({
    shouldShowAlert: true,
    shouldPlaySound: true,
    shouldSetBadge: false,
  }),
});

export default function App() {
  const [expoPushToken, setExpoPushToken] = useState('');
  const [message, setMessage] = useState('');

  useEffect(() => {
    registerForPushNotificationsAsync().then(token => setExpoPushToken(token));
  }, []);

  async function registerForPushNotificationsAsync() {
    let token;

    if (Platform.OS === 'android') {
      await Notifications.setNotificationChannelAsync('default', {
        name: 'default',
        importance: Notifications.AndroidImportance.MAX,
      });
    }

    if (Device.isDevice) {
      const { status: existingStatus } = await Notifications.getPermissionsAsync();
      let finalStatus = existingStatus;

      if (existingStatus !== 'granted') {
        const { status } = await Notifications.requestPermissionsAsync();
        finalStatus = status;
      }

      if (finalStatus !== 'granted') {
        Alert.alert('Failed to get push token for push notification!');
        return;
      }
```

```
      // Get the token
      token = (await Notifications.getExpoPushTokenAsync()).data;
      console.log('My Token:', token);
    } else {
      Alert.alert('Must use physical device for Push Notifications');
    }

    return token;
  }

  const sendNotification = async () => {
    try {
      // Replace with your backend URL
      await axios.post('http://10.0.2.2:3000/send-notification', {
        token: expoPushToken,
        message: message,
      });
      Alert.alert('Success', 'Request sent to backend!');
    } catch (error) {
      console.error(error);
      Alert.alert('Error', 'Could not send notification');
    }
  };

  return (
    <View style={styles.container}>
      <Text style={styles.title}>Push Notification System</Text>

      <Text style={styles.label}>Your Token:</Text>
      <Text style={styles.token}>{expoPushToken ? expoPushToken.slice(0, 20) + '...' : 'Loading...'}</Text>

      <TextInput
        style={styles.input}
        placeholder="Enter notification message"
        value={message}
        onChangeText={setMessage}
      />
      <Button title="Trigger Notification via Backend" onPress={sendNotification} />
    </View>
  );
}

const styles = StyleSheet.create({
  container: { flex: 1, alignItems: 'center', justifyContent: 'center', padding: 20 },
  title: { fontSize: 20, fontWeight: 'bold', marginBottom: 20 },
  label: { fontSize: 16, marginBottom: 5 },
  token: { fontSize: 12, color: 'gray', marginBottom: 20 },
  input: { height: 40, width: '100%', borderColor: 'gray', borderWidth: 1, marginBottom: 20,
           paddingHorizontal: 10,
}
});
```

Output:


Backend Triggered Notification!
This is a test notification from Node.js
Experiment 8
Push Notification System
ExponentPushToken[xxxxxxxxxxpNCOHOC,7x...
Enter notification message
Trigger Notification via Backend

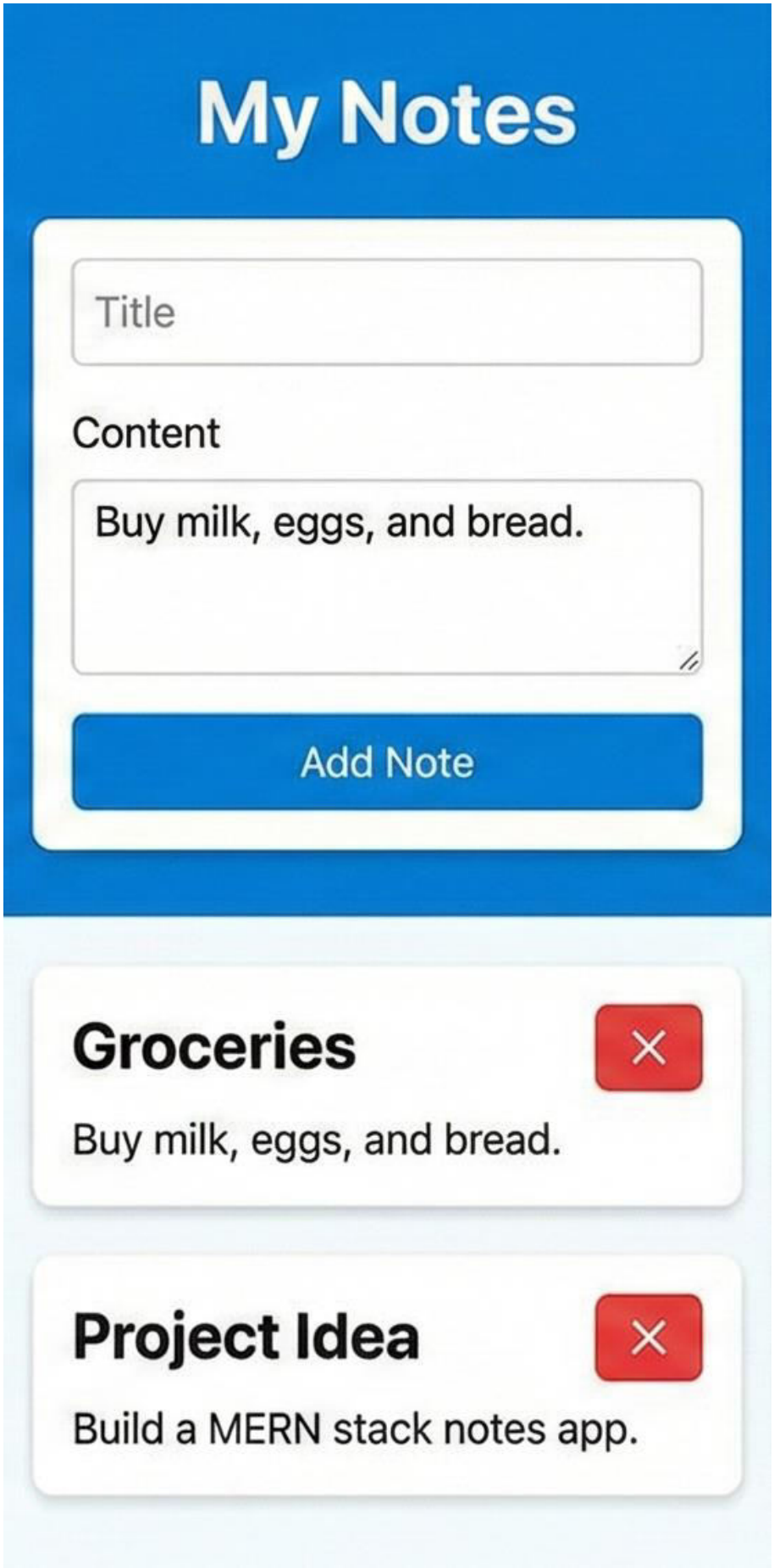
Experiment - 9

Aim : Build a simple Notes App where:
Backend stores notes in MongoDB.
React Native frontend allows creating, viewing, and deleting notes.
Key Focus: End-to-end integration, CRUD, Axios, UI flow.

Code :

```
const express = require('express');
const mongoose = require('mongoose');
const cors = require('cors');
const bodyParser = require('body-parser');

const app = express();
app.use(cors());
app.use(bodyParser.json());

// 1. Connect to MongoDB
mongoose.connect('mongodb://127.0.0.1:27017/notes_app_db')
  .then(() => console.log('Connected to MongoDB'));

// 2. Define Schema
const noteSchema = new mongoose.Schema({
  title: String,
  content: String
});
const Note = mongoose.model('Note', noteSchema);

// 3. API Routes

// GET: Fetch all notes
app.get('/notes', async (req, res) => {
  const notes = await Note.find();
  res.json(notes);
});

// POST: Create a new note
app.post('/notes', async (req, res) => {
  const newNote = new Note(req.body);
  await newNote.save();
  res.json(newNote);
});

// DELETE: Remove a note
app.delete('/notes/:id', async (req, res) => {
  await Note.findByIdAndDelete(req.params.id);
  res.json({ message: 'Note deleted' });
});

app.listen(3000, () => console.log('Server running on port 3000'));
```

Output :


My Notes
Title
Content
Buy milk, eggs, and bread.
Add Note
Groceries
Buy milk, eggs, and bread.
Project Idea
Build a MERN stack notes app.

Experiment - 10

Aim : Create a mobile app that:
Lets the user pick an image using ImagePicker
Uploads the image to your Express backend (using multipart/form-data)
Backend must:
Store the image in MongoDB GridFS or local file storage
Return the stored image URL or ID
Frontend must:
Display the uploaded image on screen
Key Focus: File upload, API integration, form-data handling.

Code :

```
//Server.js

const express = require('express');
const mongoose = require('mongoose');
const cors = require('cors');
const multer = require('multer');
const path = require('path');

const app = express();
app.use(cors());
app.use(express.json());

// 1. Serve Static Files (to allow access to uploaded images)
// Images saved in 'uploads' folder will be accessible at http://localhost:3000/uploads/filename
app.use('/uploads', express.static('uploads'));

// 2. Configure Multer for Local File Storage
const storage = multer.diskStorage({
  destination: (req, file, cb) => {
    cb(null, 'uploads/'); // Make sure this folder exists
  },
  filename: (req, file, cb) => {
    // Save file with unique name (timestamp + original name)
    cb(null, Date.now() + '-' + file.originalname);
  }
});

const upload = multer({ storage: storage });

// 3. Connect to MongoDB
mongoose.connect('mongodb://127.0.0.1:27017/image_upload_db')
  .then(() => console.log('Connected to MongoDB'));

// Mongoose Schema to store image metadata
const imageSchema = new mongoose.Schema({
  imageUrl: String
});
const ImageModel = mongoose.model('Image', imageSchema);

// 4. Upload Route
```

```
// 'image' is the key name expected in form-data
app.post('/upload', upload.single('image'), async (req, res) => {
  try {
    if (!req.file) {
      return res.status(400).send('No file uploaded');
    }

    // Create the full URL for the image
    // NOTE: For Android Emulator, use machine IP, but for server logic 'req.protocol' works
    const imageUrl = `http://10.0.2.2:3000/uploads/${req.file.filename}`;

    const newImage = new ImageModel({ imageUrl });
    await newImage.save();

    res.json(newImage);
  } catch (error) {
    res.status(500).send(error.message);
  }
});

app.listen(3000, () => console.log('Server running on port 3000'));

//App.js

import React, { useState } from 'react';
import { StyleSheet, Text, View, Button, Image, Alert, ActivityIndicator } from 'react-native';
import * as ImagePicker from 'expo-image-picker';
import axios from 'axios';

export default function App() {
  const [selectedImage, setSelectedImage] = useState(null);
  const [uploadedImageUrl, setUploadedImageUrl] = useState(null);
  const [uploading, setUploading] = useState(false);

  // 1. Pick Image from Gallery
  const pickImage = async () => {
    // Request permissions
    const permissionResult = await ImagePicker.requestMediaLibraryPermissionsAsync();

    if (permissionResult.granted === false) {
      Alert.alert("Permission Required", "You need to allow access to your photos!");
      return;
    }

    const result = await ImagePicker.launchImageLibraryAsync({
      mediaTypes: ImagePicker.MediaTypeOptions.Images,
      allowsEditing: true,
      aspect: [4, 3],
      quality: 1,
    });

    if (!result.canceled) {
      setSelectedImage(result.assets[0].uri);
    }
```

```
};

// 2. Upload Image to Backend
const uploadImage = async () => {
  if (!selectedImage) {
    Alert.alert('No image selected');
    return;
  }

  setUploading(true);

  // Prepare Form Data
  const formData = new FormData();
  formData.append('image', {
    uri: selectedImage,
    name: 'photo.jpg',
    type: 'image/jpeg',
  });

  try {
    const response = await axios.post('http://10.0.2.2:3000/upload', formData, {
      headers: {
        'Content-Type': 'multipart/form-data',
      },
    });

    // Update state with the URL received from backend
    setUploadedImageUrl(response.data.imageUrl);
    Alert.alert('Success', 'Image Uploaded!');
  } catch (error) {
    console.error(error);
    Alert.alert('Error', 'Upload failed');
  } finally {
    setUploading(false);
  }
};

return (
  <View style={styles.container}>
    <Text style={styles.title}>Image Upload App</Text>

    {/* Button to Pick Image */}
    <Button title="Pick an image from camera roll" onPress={pickImage} />

    {/* Preview Selected Image */}
    {selectedImage && (
      <Image source={{ uri: selectedImage }} style={styles.image} />
    )}

    {/* Button to Upload */}
    <View style={{ marginTop: 20 }}>
      {uploading ? (
        <ActivityIndicator size="large" color="#0000ff" />
      ) : (
```

```
        <Button title="Upload Image" onPress={uploadImage} disabled={!selectedImage} />
      )}
    </View>

    {/* Display Uploaded Image from Server */}
    {uploadedImageUrl && (
      <View style={styles.resultContainer}>
        <Text style={styles.label}>Uploaded to Server:</Text>
        <Image source={{ uri: uploadedImageUrl }} style={styles.image} />
      </View>
    )}
  </View>
);
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    alignItems: 'center',
    justifyContent: 'center',
    padding: 20,
    backgroundColor: '#fff'
  },
  title: {
    fontSize: 24,
    fontWeight: 'bold',
    marginBottom: 20
  },
  image: {
    width: 200,
    height: 200,
    marginTop: 10,
    borderRadius: 10,
    borderWidth: 1,
    borderColor: '#ddd'
  },
  resultContainer: {
    marginTop: 30,
    alignItems: 'center'
  },
  label: {
    fontSize: 16,
    fontWeight: 'bold',
    marginBottom: 5,
    color: 'green'
  }
});
```

Output :

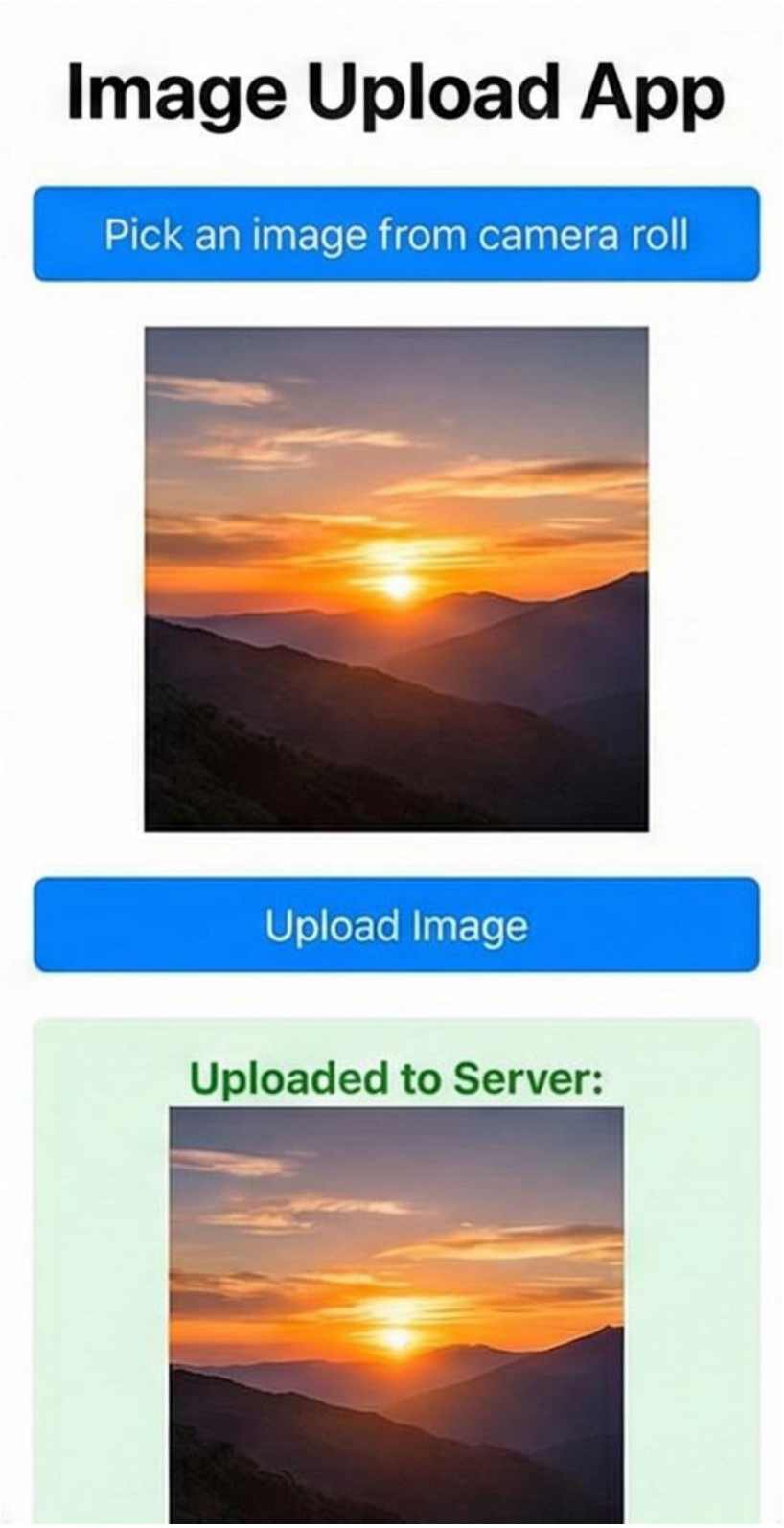


Image Upload App
Pick an image from camera roll
Upload Image
Uploaded to Server: